

"Higher order Functions - Map"

هذا الـ Function الذي يتقبل Function آخرى كـ Parameter والـ Function التالية دي فيها Parameters كدري
 النوع الأول للـ Higher order Function "Return New" **Map** **Array**

عبارة عن **Array Method** **create a new array**

Syntax: `map(CallBackFunction (Element, Index, Array) this Arg 1).`

Element: The current Element being Processed in the array.

Index: The Index of the current element being Processed in the array.

Array: The current Array.

لو عندك Array وكاوز أرجع y Array ثانية عبارة عن نفس عدد العناصر **اختياري**

لا y array الأول ولكن قيمة كل عنصر من الـ Array الثانية ضعه
 قيمة العنصر من الـ Array الأول فايدل ما أكمل كده بـ Loop ممكن

استخدم `map()` كالتالي: `Let myNums = [1, 2, 3, 4, 5, 6];`

`Let addSelf = myNums.map(Function(element) {
 return element + element;`

`3, 10);`

`this === 10;`

ويمكن افول الـ Regular إلى Arrow

`map(element) => element + element;`

ويمكن عمل Function واستخدمها في الـ map باسمها:

`Function addition(ele) {
 return ele + ele;`

`}`

`map(addition);`

Higher order Function - Map Practice

Map

- Swap Cases.
- Inverted Numbers.
- Ignore Boolean Value.

```
1. Let swappingCases = "e1ZERo";
   Let sw = swapping .. split(" ").map(Function (ele) {
       return ele === ele.toUpperCase() ?
           ele.toLowerCase() : ele.toUpperCase();
   }).join(" ");
   console.log(sw);
```

```
2. Let invertedNumbers = [1, -10, -20, 15, 100, -30];
   Let inv = inverted .. map(Function (ele) {
       return -ele;
   });
   console.log(inv);
```

```
3. Let ignoreNumbers = "E1Z123er4o";
   Let ign = ignore .. map(Function (ele) {
       return isNaN(split(" ")parseInt(ele)) ? ele : "";
   }).join(" ");
   console.log(ign);
```

Higher order Functions - Filter

نفس ال Syntax يتبع map ولكن سيمر بانه يرجع array جديده فيها العناصر اللي عدت من ال test.

يعني بقط Condition لو true
هيدري وينقط من ال array لو False مش هيصفيه

Example:

```
Let Friends = ["Ahmed", "sameh", "sayed", "Amged", "Israa"]
```

```
Let FilterFriends = Friends.Filter(Function (el) {  
    return el.startsWith("A")? true: false;
```

3);

له مش لازم اكتب

```
console.log(FilterFriends);
```

* Higher order Functions - Filter Practice *

#074

Filter Longest word By Number.

// Filter words more than 4 char.

```
let sentence = "I Love Food Code Too Playing  
Much";
```

```
let smallWords = sentence.split(" ").filter(function (el)
```

```
{  
  return ele.length < 5;
```

```
}).join(" ");
```

```
console.log(smallWords);
```

"Higher order Functions - Reduce"

يستخدم ال Function على كل عنصر من عناصر ال Array و
 map و Filter ولكن بتعقل تحقق النتائج لغاية ما
 ترجع قيمة واحدة بس.

Reduce بتقبل حاجتين هما ال Call Back Function و ال
 Initial Value

ال Callback Function بتقبل أربع حاجات وهما:

(Accumulator - current Value - Index - Array)
 Example: ↓ Element ↓ current ↓ nums
↓ array

Let nums = [5, 10, 20, 15, 30];

Let add = nums.reduce (Function (acc, current, index
 arr) {

return acc + current ;

5) ; ↓ initial Value ↓ القيمة اللي بتبدأ

console.log (add); ↓ ال acc اللي هو

ال acc

* #076

* Higher order Functions - Reduce Practice

Let removeChars = ["E", "@", "@", "L", "Z", "@", "E", "R", "O"];

Let finalString = removeChars.filter(function (el) {

return !~~st~~el.startsWith("@");

}).reduce(function (acc, current) {

return `\${acc} \${current}`;

});

console.log(finalString); → ELZER0.

*

*

يُفضل استخدامها لما تكون مش عاوز أغير من عناصر ال array

الموضوع: #77

التاريخ: ٢٠٢٦/٨/٢٦

"Higher order Functions - ForEach And Practice"

الانتش array جديدة .
method executes a provided Function once for each array Element.

Note:

- Doesn't Return Anything [undefined].
- Break will Not Break The Loop.

Syntax:

ForEach (callback Function (Element, Index, Array) this Arg);

- Element: The current Element being Processed in the Array.
- Index: The Index of the current Element being Processed in the Array.
- Array: The current Array.

Example:

ul > 3 * Li

HTML

+ div. content > 3 * div

JS

```
Let allLis = document.querySelectorAll("ul")
allLis.forEach (Function (ele) {
```

```
  ele.onclick = Function () {
```

```
    // Remove Active Class From All Elements.
```

```
    allLis.forEach (Function (ele) {
```

```
      ele.classList.remove ("active");
```

```
    });
```

```
    // Add Active class To this Element
```

```
    this.classList.add ("active");
```

```
  });
```